

# SKIVE on gpt-oss: the integration, problem by problem, change by change

This is the reference document for the gpt-oss integration of SKIVE into vLLM 0.23.0. It is written for someone who will maintain or extend the code. It defines every term it uses, states each problem gpt-oss created, shows the upstream code that was there before, shows the inserted code, explains why the change is correct, and ends with the verification evidence. All "before" code is quoted verbatim from vLLM v0.23.0; all "after" code is what the fork contains today (`skive/patches/*.patch` are the exact diffs).

**Files referenced.** Engine hooks: `vllm/config/cache.py`, `vllm/engine/arg_utils.py`, `vllm/v1/worker/gpu_model_runner.py`, `vllm/v1/engine/core.py`, `vllm/v1/attention/backends/flash_attn.py`, `vllm/v1/attention/backends/triton_attn.py`, `vllm/v1/worker/gpu/model_runner.py`. SKIVE package: `vllm/kv_evict/{integration,selection,scoring,compaction,manager,fused_attention}.py`. Model: `vllm/model_executor/models/gpt_oss.py`. Backend selection: `vllm/platforms/cuda.py`, `vllm/v1/attention/backends/fa_utils.py`, `vllm/utils/flashinfer.py`.

## 1. Definitions

- **KV cache.** For every past token and every attention layer, the model stores a key vector K and a value vector V. Attention for a new token reads all of them. This store grows with every token of every request and is the dominant GPU memory consumer during decoding.
- **Prefill / decode.** Prefill processes the prompt in one pass and fills the cache. Decode generates one token per step, appending to the cache. SKIVE only evicts during decode.
- **Block (page).** vLLM does not store the cache contiguously. GPU memory is divided into fixed-size blocks of `block_size` tokens (16 by default). A block holds the K and V of 16 consecutive tokens for one layer group.
- **Block table.** Per request, the ordered list of physical block ids that hold its tokens. Logical token position `i` lives in `table[i // block_size]`. The attention kernel receives this table and gathers keys through it. The worker keeps it as a NumPy array (`input_batch.block_table[g].block_table.np`) and mirrors it to a GPU tensor for the kernel.
- **Null block.** vLLM reserves physical block id 0 as a placeholder that is never written and reads as zeros. SKIVE repoints evicted slots at it; vLLM's own sliding-window manager does the same for out-of-window tokens.
- **KV-cache group.** vLLM groups attention layers by the kind of cache they need. Every full-attention layer goes in one group; every sliding-window layer with the same window goes in another. Each group has its own block size, its own per-request block table on the worker, and its own **single-type manager** on the scheduler that allocates and frees blocks. A model with one attention type has one group; gpt-oss has two.
- **Scheduler and worker.** vLLM's V1 engine runs a scheduler (decides which requests run, owns block allocation, in the `EngineCore` process) and one worker per GPU (runs the model, owns the model runner and the block tables it hands to kernels). They are different processes; the scheduler reaches the worker through `collective_rpc`.
- **Model runner.** The worker object that prepares inputs, runs the forward pass, samples, and produces outputs each step (`GPUModelRunner.execute_model`). vLLM 0.23 has a V1 runner and a newer V2

runner; SKIVE hooks the V1 runner.

- **Attention backend.** The kernel implementation vLLM uses for attention: FlashAttention (versions 2, 3, 4), FlashInfer, the Triton unified kernel, FlexAttention, and others. Each backend has an **implementation class** (runs the kernel) and a **metadata builder** (turns the batch description into the kernel's arguments every step).
- **Attention metadata.** The per-step arguments of the attention kernel: `query_start_loc` (where each request's query tokens start in the flattened batch), `seq_lens` (how many cached tokens each request has), `block_table` (the gather map), `slot_mapping` (where new K/V are written), `max_seq_len`, and so on.
- **Attention sink.** In most transformers the first tokens attract a large share of attention mass regardless of content. gpt-oss makes this explicit: every attention head has a **learned sink logit** that participates in the softmax denominator, so a head can route attention "nowhere" instead of over real tokens. In vLLM this is the `sinks` parameter of the attention layer, passed to FlashAttention-3 as `s_aux`.
- **Grouped-query attention (GQA).** Several query heads share one K/V head. gpt-oss has 64 query heads and 8 KV heads, so 8 query heads read each cached K/V.
- **Sparse-gather.** SKIVE's technique of removing evicted (null) slots from the block table the kernel sees and shortening the request's key length accordingly, so the kernel never reads evicted blocks.
- **Budget, sink blocks, local blocks.** `kv_evict_budget` is the maximum number of real (non-null) blocks a request may keep. The first `kv_evict_num_sink_blocks` and the last `kv_evict_num_local_blocks` real blocks are never evicted.
- **Eviction, reclaim.** Eviction (worker side) repoints a block-table slot at the null block. Reclaim (scheduler side) returns the physical block to the pool so it can be allocated to another request. Only reclaim frees memory.
- **Importance metric.** The score that ranks blocks for eviction. `vk_ratio` is static: the L2 norm of the block's values over the norm of its keys, computed once per block and cached. `value_attention` is query-dependent: the attention weight of the current decode query on each token times the L1 norm of that token's value, summed over the block. `h2o` and `snapkv` are attention-only baselines.
- **Harmony format.** gpt-oss's output format: an `analysis` channel (reasoning) followed by a `final` channel (the answer), delimited by special tokens such as `<|channel|>final<|message|>`. The chat template takes a `reasoning_effort` argument (low, medium, high).
- **MXFP4.** The 4-bit block floating-point format of gpt-oss's expert weights. On Ada GPUs vLLM runs them with the Marlin kernels.
- **CUDA graphs, piecewise.** A CUDA graph replays a recorded sequence of kernels without CPU launch overhead. vLLM can capture the whole decode step (full) or only the pieces between attention calls (piecewise), leaving attention eager. Captured graphs require every tensor they touch to keep its address.
- **cross%, eviction rate, KV saved.** Evaluation columns: the share of requests that evicted at least one block; evicted cache tokens per generated token; evicted tokens over all tokens.

## 2. SKIVE before gpt-oss: what existed and how it worked

SKIVE was built and validated on models with one attention type in every layer (Qwen2.5, Llama, DeepSeek-R1-Distill, Phi-3.5). Its design has four rules: hook-level integration (it edits block tables inside the engine, which no wrapper can do), all logic in one package with thin call-outs in vLLM, off by default with byte-identical behaviour when off, and no kernel changes.

**The hooks (edits A to J, thirteen insertions in six files).** Edits A to D add four engine arguments (`kv_evict_enabled`, `kv_evict_budget`, `kv_evict_num_sink_blocks`, `kv_evict_num_local_blocks`) to `CacheConfig` and `EngineArgs` and the CLI. Edit E calls `skive_post_step(model_runner)` once per decode step in the worker, after outputs are built. Edits F and G, in the scheduler process, fetch the list of blocks the worker evicted and free them. Edit H makes the V2 runner refuse eviction loudly. Edits I and J, in the FlashAttention backend, capture the decode query for query-dependent metrics and apply sparse-gather in the metadata builder.

### One decode step with SKIVE (before gpt-oss).

1. The attention forward stores the layer's query tensor on the layer object (`layer._skive_q`).
2. After the step, `skive_post_step` reads the worker's block table, counts real blocks per row in one vectorized NumPy operation, and finds rows over budget.
3. For those rows it scores blocks (cached `vk_ratio`, or one batched `value_attention` computation for all rows with a single GPU-to-CPU synchronization) and nulls the lowest-scoring non-protected slots.
4. It records (`request_id`, `logical_index`) pairs on the runner.
5. In the scheduler process, edits F/G pull those pairs and free the physical blocks in the manager.
6. On the next step, the FlashAttention metadata builder compacts the block table so the kernel skips the nulled slots.

**The two assumptions this design made, both wrong for gpt-oss:** there is exactly one KV-cache group (so index 0 is "the" block table and "the" manager, and `model_runner.kv_caches[layer_index]` is the layer's cache), and the attention backend is FlashAttention (so the capture and compaction hooks in that file always run).

## 3. gpt-oss: the architecture facts, with code

From `vllm/model_executor/models/gpt_oss.py` (the model implementation vLLM uses):

```
# OAIAttention.__init__ (abridged)
self.sinks = torch.nn.Parameter(torch.empty(config.num_attention_heads // tp_size, ...))
self.sliding_window = config.sliding_window if layer_idx % 2 == 0 else None
self.attn = Attention(self.num_local_attention_heads, self.head_dim, self.scaling,
                      num_kv_heads=self.num_local_key_value_heads, cache_config=cache_config,
                      quant_config=quant_config, per_layer_sliding_window=self.sliding_window,
                      attn_type=AttentionType.DECODER, prefix=f"{prefix}.attn",
                      sinks=self.sinks)
```

- **Alternating attention.** Even layers get `sliding_window = 128`; odd layers get `None` (full attention). vLLM therefore builds a `SlidingWindowSpec` group for even layers and a `FullAttentionSpec` group for odd layers.

- **Learned sinks.** `self.sinks` is a parameter of shape `[num_heads]`, one logit per head, passed into `Attention(..., sinks=...)`.
- **Heads.** 64 query heads, 8 KV heads, head size 64 (from the model card, arXiv 2508.10925).
- **Experts.** A `FusedMoE` block per layer with MXFP4 weights (`_load_weights_mxfp4`); 32 experts top-4 in the 20b model.

From the attention backends, the sink support is narrow:

```
# vllm/v1/attention/backends/fa_utils.py
def flash_attn_supports_sinks() -> bool:
    if current_platform.is_xpu():
        return True
    return get_flash_attn_version() in (3, 4)

# vllm/v1/attention/backends/flash_attn.py, FlashAttentionImpl.__init__
if self.sinks is not None:
    assert flash_attn_supports_sinks(), "Sinks are only supported in FlashAttention 3"
```

and `get_flash_attn_version()` picks version 3 only when `device_capability.major == 9` (Hopper) and version 4 only when `major == 10` (datacenter Blackwell); every other GPU gets version 2, which has no sinks. FlashInfer's `supports_sink()` returns `supports_trtllm_attention()`, which requires `is_device_capability_family(100)`. The Triton backend's `supports_sink()` returns `True` unconditionally. With the priority order in `vllm/platforms/cuda.py` (`FLASH_ATTN`, `FLASHINFER`, `TRITON_ATTN`, `FLEX_ATTENTION`), the selector on an L40S (capability 8.9) or an RTX PRO 6000 (12.0) rejects the first two for a model with sinks and lands on `TRITON_ATTN`. The log line on the box confirms it: `Using AttentionBackendEnum.TRITON_ATTN backend.`

## 4. The problems, and the solution to each

### Problem 1: two KV-cache groups, and index 0 is not the right one

**What broke.** With two groups, a request has two block tables on the worker (`input_batch.block_table[0]` and `[1]`) and two managers on the scheduler (`coordinator.single_type_managers[0]` and `[1]`). The old code used index 0 everywhere. vLLM orders groups by its own grouping logic, not by "full attention first"; on the box the full-attention group resolved as **group 1**. Evicting from group 0 would have nulled slots of the sliding-window table (where vLLM already nulls out-of-window slots itself) and freed the wrong physical blocks. Worse, the old scorer walked `model_runner.kv_caches[li]` for every layer and indexed each with the group-0 block ids, so half the layers would have been scored with block ids from the other group's table.

**Before** (integration.py, old):

```
bt = model_runner.input_batch.block_table[0]
...
for li, kc in enumerate(model_runner.kv_caches):    # every layer, group-0 ids
    kb = kc[idx, 0]; vb = kc[idx, 1]
...
mgr = kv_cache_manager.coordinator.single_type_managers[0]    # reclaim
```

**After.** A resolution layer, cached on the runner:

```
_NON_FULL_SPEC_HINTS = ("SlidingWindow", "Mamba", "Cross", "EncoderOnly", "ChunkedLocal", "UniformType")

def _is_full_attention_spec(spec) -> bool:
    name = type(spec).__name__
    if any(h in name for h in _NON_FULL_SPEC_HINTS):
        return False
    return (getattr(spec, "sliding_window", None) is None
            and getattr(spec, "attention_chunk_size", None) is None)

def _skive_kv_groups(model_runner):    # [(gid, spec, layer_names)]
    try:
        return [(i, g.kv_cache_spec, list(g.layer_names))
                for i, g in enumerate(model_runner.kv_cache_config.kv_cache_groups)]
    except Exception:
        ... fall back to model_runner.attn_groups (dedupe by kv_cache_group_id) ...

def _skive_full_group(model_runner):    # -> (gid, layer_names | None)
    groups = _skive_kv_groups(model_runner)
    if groups is not None and len(groups) > 1:
        pick = first group whose spec is pure full attention
        if pick is None: pick = first group whose spec class name contains "FullAttention" or "MLAAttention"
        if pick is None: model_runner._skive_multigroup_unresolved = True
    ...
```

Everything downstream uses it: `_skive_block_table` returns `input_batch.block_table[gid]` (or `None`, with one warning, when unresolved, so **no eviction happens rather than a wrong one**); `_skive_layers` returns only the attention modules whose names are in the group; `_layer_kv(layer)` reads the layer's own cache tensor `layer.kv_cache`, the one vLLM's `bind_kv_cache` attached at start-up, instead of indexing a shared list by position; pending frees are `(request_id, logical_index, gid)` triples and `skive_reclaim` frees in `managers[gid]`. The sliding-window group is never touched: vLLM's own `SlidingWindowManager.remove_skipped_blocks` already bounds it at 128 tokens.

**Why duck typing.** The spec classes are matched by name and attributes instead of `isinstance`, so `integration.py` needs no `vLLM` import (it must be importable inside the worker before `vLLM` finishes initializing) and can be unit-tested with stubs.

**Guard.** If kernel and manager block sizes differ (`bt.use_hybrid_blocks`), worker logical indices would not equal manager indices at reclaim; eviction is disabled with one warning.

## Problem 2: the sink changes the softmax the model actually computes

**What broke.** With a sink logit `s_h`, the attention weight of token `t` in head `h` is  $\exp(l_t) / (\exp(s_h) + \sum_t \exp(l_t))$ . The old scorer used `torch.softmax(logits)`, whose denominator lacks the sink term. Within one head that only rescales all weights equally, so the ranking of tokens within a head is unchanged (a unit test checks this), but across heads the weights are then combined, and heads with a large sink should contribute less. Scoring must match the distribution the model uses.

**Before:** `p = torch.softmax(logits, dim=2).`

**After:**

```
def _layer_sinks(layer):
    s = getattr(getattr(layer, "impl", None), "sinks", None)
    return s if isinstance(s, torch.Tensor) else None

def _softmax_with_sink(logits, sinks):          # logits [R, Hq, T], masked entries -inf
    if sinks is None:
        return torch.softmax(logits, dim=2)
    R, Hq, _ = logits.shape
    s = sinks.to(logits.device, torch.float32).view(1, Hq, 1).expand(R, Hq, 1)
    return torch.softmax(torch.cat([logits, s], dim=2), dim=2)[: , :, :-1]
```

The sink is appended as one extra column before the softmax and dropped afterwards, which is exactly the FlashAttention-3 `s_aux` semantics. `layer.impl.sinks` exists on both `FlashAttentionImpl` and `TritonAttentionImpl`, so this works on both backends.

## Problem 3: 64 query heads on 8 KV heads

**What broke.** A block's importance is aggregated over 64 query heads. Raw per-head scores differ in scale by orders of magnitude; a plain sum lets a few heads decide everything.

**Before:** `token_score = per_head.sum(dim=1).`

**After:** `_agg_heads(x, valid, mode)` with `SKIVE_HEAD_AGG = sum | max | zmax`. `zmax` standardizes each head's scores over that head's valid tokens (z-score), then takes the maximum across heads: a token is kept if any head considers it unusually important. This is the aggregation TriAttention uses for GQA (arXiv 2604.04921, section 4.3). Padding tokens are masked before the statistics and zeroed after.

## Problem 4: the hooks lived only in the FlashAttention backend

**What broke.** On the L40S, gpt-oss runs on `TRITON_ATTN` (section 3). Edits I and J were in `flash_attn.py`, which is never instantiated for gpt-oss there. Consequences: `layer._skive_q` was never set, so `value_attention` silently fell back to `vk_ratio` (the scorer returns `None` when the query is missing); and no compaction happened, so evicted blocks were still gathered by the kernel as zero keys, which both wastes the read and dilutes the softmax with `exp(0)` terms. Eviction and reclaim themselves still worked, because

they do not depend on the backend.

The fix, edit K, is the subject of section 5.

## Problem 5: harmony output and reasoning effort

**What broke.** The raw completion contains the analysis channel, control tokens and then the final channel. Scoring the raw text as the answer is wrong (a boxed number inside the reasoning could be a wrong intermediate value), and the reasoning effort must be passed through the chat template or the model runs at its default.

**Fix (harness).** Decode with `skip_special_tokens=False`, then:

```
_HARMONY_TOK = re.compile(r"<\[a-z_\]+\>")
def extract_final(text):
    for marker in ("<|channel|>final<|message|>", "assistantfinal"):
        if marker in text:
            text = text.split(marker)[-1]; break
    return _HARMONY_TOK.sub("", text).strip()
```

and `llm.chat(..., chat_template_kwargs={"reasoning_effort": REASONING})` with a system-prompt fallback if the template rejects the argument.

## Problem 6: engine plumbing that only surfaced on gpt-oss

- **Serialization of the eviction counter.** The harness reads per-request eviction counts with `llm.collective_rpc(callable).vLLM's` multiprocessing engine encodes RPC arguments with msgpack and refuses callables unless `VLLM_ALLOW_INSECURE_SERIALIZATION=1`:

```
# vllm/v1/serial_utils.py (upstream)
if not envs.VLLM_ALLOW_INSECURE_SERIALIZATION:
    raise TypeError(f"Object of type {type(obj)} is not serializable ...")
```

The runners set the flag before the engine starts. Without it, cross% and eviction rate were blank ("n/a (TypeError)"). - **Counting evictions per request.** The worker's request ids are not the frontend's `RequestOutput.request_id`, and the warm-up request also evicts. The harness snapshots the counter after warm-up and counts only keys that appear afterwards. - **V1 runner.** `VLLM_USE_V2_MODEL_RUNNER=0`; edit H raises otherwise. - **Prefix caching off.** Reclaim frees a block directly; with prefix caching a block can be shared by several requests through reference counts, which reclaim does not handle.

## Problem 7: eviction forced eager mode and paid for it

**What broke.** Sparse-gather rewrites the block table tensor every step inside the metadata builder. Under a fully captured CUDA graph the kernel arguments must keep their addresses, so the first gpt-oss runs used `enforce_eager=True`, and on this decode-bound 20B MoE that cost more than eviction saved (tpot 50 to 64 ms).

**Fix.** Piecewise CUDA graphs: `enforce_eager=False` with `CompilationConfig(cuda_graph_mode=CUDAGraphMode.PIECEWISE)`. Attention stays eager (so the builder may hand it fresh tensors) while the expert and MLP kernels are replayed. The Triton hook additionally skips compaction when `self.decode_cuda_graph_enabled` is true, which is the case only for the full-capture modes.

Result: tpot 51 ms with eviction versus 50 ms FullKV eager.

## Problem 8 (AIME): the prompt itself was evictable

**What broke.** The sink protection is 2 blocks (32 tokens). An AIME prompt spans about 8 blocks. Once the reasoning trace exceeded the budget, most of the problem statement competed with reasoning blocks on the importance score and was regularly evicted; the model then re-derived and looped.

**Before** (`evict_request_blocks`): `_plan_eviction(row[:n], cfg.num_sink_blocks, cfg.num_local_blocks, cfg.kv_budget)`.

**After:**

```
sink = cfg.num_sink_blocks
if _SKIVE_PROTECT_PROMPT:
    try: # V1 InputBatch keeps the prompt length per row
        npt = int(model_runner.input_batch.num_prompt_tokens[req_index])
        sink = max(sink, -(-npt // int(bt.block_size)) + 1)
    except Exception:
        pass
real_indices, candidates_k, num_to_evict = _plan_eviction(row[:n], sink, cfg.num_local_blocks, cfg.kv_budget)
```

Effect at a 2048-token budget: 49.2 to 65.0 pass rate at the same 77 percent of KV evicted. It is a switch (`SKIVE_PROTECT_PROMPT=1`) because on long-document tasks the prompt is what must be evicted.

## 5. The Triton backend integration in depth

### 5.1 What the upstream Triton backend does

`vllm/v1/attention/backends/triton_attn.py` has three parts. `TritonAttentionMetadata` is a dataclass of kernel arguments. `TritonAttentionMetadataBuilder.build()` fills it every step from the engine's `CommonAttentionMetadata`:

```
# upstream build(), abridged
num_actual_tokens = common_attn_metadata.num_actual_tokens
max_query_len = common_attn_metadata.max_query_len
max_seq_len = common_attn_metadata.max_seq_len
query_start_loc = common_attn_metadata.query_start_loc
seq_lens = common_attn_metadata.seq_lens
block_table_tensor = common_attn_metadata.block_table_tensor
slot_mapping = common_attn_metadata.slot_mapping

use_cascade = common_prefix_len > 0
...
attn_metadata = TritonAttentionMetadata(num_actual_tokens=..., query_start_loc=query_start_loc,
                                       seq_lens=seq_lens, block_table=block_table_tensor, slot_mapping=slot_mapping, ...)
```

`TritonAttentionImpl.forward()` writes the new K/V into the cache (`do_kv_cache_update`, separately) and calls the kernel:

```
unified_attention(q=query[:num_actual_tokens], k=key_cache, v=value_cache, out=output[:num_actual_tokens],
                 cu_seq_lens_q=cu_seq_lens_q, max_seq_len_q=max_seq_len_q, seqused_k=seqused_k, max_seq_len_k=max_seq_len_k,
                 softmax_scale=self.scale, causal=True, window_size=self.sliding_window, block_table=block_table,
                 ..., sinks=self.sinks, ...)
```

The two arguments that matter for SKIVE are `block_table` (the gather map: for request `r`, key position `t` is read from physical block `block_table[r, t // block_size]`) and `seqused_k` (`seq_lens`: how many key positions to read for request `r`). This is the same contract FlashAttention's `flash_attn_varlen_func` uses, which is why the FlashAttention hook ports one to one. The builder is instantiated with the group's `kv_cache_spec` and the engine's `vllm_config`, and it records `self.decode_cudagraph_enabled` (true for the full-capture modes).

## 5.2 Where the two hooks go

Query capture goes at the top of `TritonAttentionImpl.forward`, before any early return:

```
# SKIVE: capture current query for value x attention eviction scoring.
if _SKIVE_CAPTURE_Q:
    layer._skive_q = query.detach()
```

`query` is `[num_tokens, num_heads, head_size]` for the whole batch; `skive_post_step` later selects each request's last query row through `query_start_loc`. `_SKIVE_CAPTURE_Q` is read once at import (`SKIVE_METRIC` in `("value_attention", "h2o", "snapkv")`), so the cost is a single attribute store per layer per step and zero when the metric is static.

**Sparse-gather** goes in `build()` immediately after the tensors are read from the common metadata and before they are placed into `TritonAttentionMetadata`:

```
if (_SKIVE_SPARSE and getattr(self.vllm_config.cache_config, "kv_evict_enabled", False)
    and not self.decode_cudagraph_enabled
    and _skive_can_compact(self.kv_cache_spec)):
    from vllm.kv_evict.compaction import compact_block_table_torch
    _n = common_attn_metadata.num_reqs
    _q_lens = query_start_loc[1 : _n + 1] - query_start_loc[:_n]
    block_table_tensor, seq_lens = compact_block_table_torch(
        block_table_tensor, seq_lens, self.block_size, query_lens=_q_lens)
```

Three guards, each with a reason. `kv_evict_enabled`: off means byte-identical upstream behaviour. `not self.decode_cudagraph_enabled`: under full decode capture the metadata tensors must be the persistent buffers, and compaction returns new tensors; piecewise graphs and eager mode are unaffected. `_skive_can_compact(spec)`: only the full-attention group may be compacted, because the sliding-window group's null slots are vLLM's own window markers and removing them would shift the window the kernel expects.

## 5.3 How compaction works, and why it is correct

`compact_block_table_torch` in `vllm/kv_evict/compaction.py`, for a batch of `R` rows:

1. `n = ceil(seq_lens / block_size)` is the number of logical blocks each row uses.
2. `is_null = (block_table == 0)` within the first `n` columns marks evicted blocks; `is_retained` is the complement.
3. Rows are selected only if they contain a null **and** have `query_len == 1` (a pure decode row). If no row qualifies, the original tensors are returned untouched (the fast path, which is what runs on every step before the first eviction).
4. For selected rows, a stable argsort on the key 0 for `retained`, 1 for `null` moves retained ids to the front in their original order; the tail is filled with 0.
5. `seq_lens` is reduced by `evicted_blocks * block_size` for those rows.

Worked example with `block_size = 16`: a decode row with `block_table = [7, 0, 9, 10]` and `seq_len = 64` becomes `[7, 9, 10, 0]` with `seq_len = 48`; the kernel then reads 48 keys through blocks 7, 9, 10. A prefill row `[3, 0, 5, 0]` with `query_len = 5` is left alone.

Why this is correct. First, attention for a single decode query does not depend on the order of the keys, only on which keys are present, so left-packing changes nothing. Second, SKIVE evicts only whole, already-full blocks outside the protected recent window, so  $\text{seq\_len} - \text{evicted} * \text{block\_size}$  is exactly the number of retained keys. Third, rotary position embeddings are applied when K is written into the cache and the query's position comes from the model's positions tensor, neither of which the gather list touches, so retained keys keep exact relative positions. Fourth, prefill rows (query length above one) keep their full table because the causal mask for multi-token queries is index-based, and that is why the `query_lens` guard exists. `max_seq_len` is not reduced; it is only a tiling upper bound and an over-estimate is safe.

## 5.4 Why this could not be done through FlashInfer or by forcing FlashAttention

Forcing `FLASH_ATTN` on an Ada GPU fails at layer construction: `assert flash_attn_supports_sinks()`. FlashInfer's sink path is the TRT-LLM kernel family, only available on SM100; on other GPUs `supports_sink()` is false and vLLM rejects the backend for gpt-oss. The Triton unified kernel supports sinks on every GPU and exposes the same gather contract, so it was both the only option on the L40S and the cheapest one to integrate.

## 5.5 The patcher

The edits are applied by `skive/patch_vllm.py`, edit K, as four anchored insertions: `import os` before `from dataclasses import dataclass`; the two flags and `_skive_can_compact` before `class TritonAttentionMetadataBuilder`; the compaction block after `slot_mapping = common_attn_metadata.slot_mapping`; the capture before `if output_block_scale is not None`. Each anchor must occur exactly once in the target file (assertion) and an already-applied edit is detected and skipped, so the patcher is idempotent and fails loudly on a different vLLM version. On 2026-09-26 it was applied to a pristine clone of v0.23.0 and the seven resulting files were byte-identical to the ones that produced the measured results.

## 6. Change-by-change reference

### 6.1 The vLLM edits (all thirteen plus edit K), with the upstream context

Each entry gives the file, the upstream lines around the insertion (before), the inserted lines (after), and the reason. The diffs in `skive/patches/` are authoritative.

**A. `vllm/config/cache.py`, `CacheConfig`.** Before: the dataclass ends its cache options with `enable_prefix_caching: bool = True` followed by `prefix_caching_hash_algo`. After: four new fields between them, `kv_evict_enabled = False`, `kv_evict_budget: int | None = None`, `kv_evict_num_sink_blocks = 0`, `kv_evict_num_local_blocks = 0`, each with a docstring. Why: the config object travels to every process, so every hook can read `cache_config.kv_evict_enabled` without any global state; defaults mean "off".

**B. `vllm/engine/arg_utils.py`, `EngineArgs`.** Before: `block_size: int | None = None` and `enable_prefix_caching: bool | None = None`. After: the same four fields defaulting to the `CacheConfig` values. Why: `LLM(**kwargs)` builds an `EngineArgs`; without the fields the kwargs are rejected.

**C. `vllm/engine/arg_utils.py`, `add_cli_args`.** Before: the cache group ends with `--kv-offloading-backend`. After: `--kv-evict-enabled`, `--kv-evict-budget`, `--kv-evict-num-sink-blocks`, `--kv-evict-num-local-blocks`. Why: `vllm serve` parity with the Python API.

**D. `vllm/engine/arg_utils.py`, `create_engine_config`.** Before: `CacheConfig(... enable_prefix_caching=self.enable_prefix_caching, prefix_caching_hash_algo=...)`. After: the four fields passed through. Why: this is the only place the config object is constructed.

**E. `vllm/v1/worker/gpu_model_runner.py`, `execute_model`.** Before: the `ModelRunnerOutput(...)` is built (ending `routed_experts=None,`) and then `if not self.use_async_scheduling:` follows. After, between them:

```
if self.cache_config.kv_evict_enabled:
    try:
        from vllm.kv_evict.integration import skive_post_step
        skive_post_step(self)
    except Exception as _skive_e:
        logger.warning("SKIVE post-step hook error: %s", _skive_e)
```

Why here: the step's sampled tokens are final, the block tables for the next step have not been built yet, and this point is reached under both synchronous and asynchronous scheduling. The try/except means a failure inside SKIVE degrades to full-cache decoding with a warning instead of killing the engine.

**F and G. `vllm/v1/engine/core.py`, `step` and `step_with_batch_queue`.** Before: each method ends with its `return engine_core_outputs, ...`. After, before the return:

```
if getattr(self.vllm_config.cache_config, "kv_evict_enabled", False):
    try:
        from vllm.kv_evict.integration import _skive_pop_pending, skive_reclaim
        _skive_pending = self.model_executor.collective_rpc(_skive_pop_pending, single_value=True)
        if _skive_pending:
            skive_reclaim(self.scheduler.kv_cache_manager, _skive_pending)
    except Exception as _skive_e:
        logger.warning("SKIVE reclaim error: %s", _skive_e)
```

Why: block allocation lives in the scheduler process; the worker cannot free a block itself.

`_skive_pop_pending` runs on the worker, returns and clears the list of `(request_id, logical_index, group_id)` triples, and `skive_reclaim` mirrors each to `null_block` in that group's manager and calls `block_pool.free_blocks`, the same path vLLM's sliding-window manager uses. Two copies because vLLM has two step loops.

**H. `vllm/v1/worker/gpu/model_runner.py`, V2 runner `__init__`.** After `self.cache_config = vllm_config.cache_config`: raise `NotImplementedError` if eviction is enabled. Why: some architectures default to the V2 runner, where the hooks do not exist; silent no-op eviction would look like a result.

**I. `vllm/v1/attention/backends/flash_attn.py`.** `import os` after `import copy`; `_SKIVE_CAPTURE_Q = os.environ.get("SKIVE_METRIC")` in `( "value_attention", "h2o", "snapkv" )` after `logger = init_logger(__name__)`; in `FlashAttentionImpl.forward`, after the version assertion and before the `output_scale` check, `layer._skive_q = query.detach()` when the flag is set. Why: query-dependent metrics need the decode query; the flag is read once so the static metric pays nothing. The gpt-oss work widened the flag from `== "value_attention"` to the three query-dependent metrics.

**J. `vllm/v1/attention/backends/flash_attn.py`.** `_SKIVE_SPARSE` flag and `_skive_can_compact` after the capture flag; in `FlashAttentionMetadataBuilder.build`, after `causal = common_attn_metadata.causal`, the compaction block guarded by `kv_evict_enabled` and `_skive_can_compact(self.kv_cache_spec)`. Why: section 5.3. The gpt-oss work added the `_skive_can_compact` gate; before it, compaction ran for every builder, which on gpt-oss would have compacted the sliding-window group.

**K. `vllm/v1/attention/backends/triton_attn.py`.** Section 5.2 and 5.5. The one difference from J is the extra `not self.decode_cudagraph_enabled` guard, because the Triton builder declares full CUDA-graph support (`AttentionCGSupport.ALWAYS`) whereas the FlashAttention-2 builder does not.

## 6.2 The `kv_evict/integration.py` changes

| function                                                                                                    | before gpt-oss                                          | after                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_skive_full_group</code> ,<br><code>_skive_kv_groups</code> ,<br><code>_is_full_attention_spec</code> | did not exist; index 0 everywhere                       | resolve the full-attention group by spec type, cache it, refuse to evict when unresolved                                                                  |
| <code>_skive_layers</code>                                                                                  | all <code>Attention</code> modules of the model         | only the modules named in the full-attention group                                                                                                        |
| <code>_layer_kv</code>                                                                                      | <code>model_runner.kv_caches[li]</code> (positional)    | <code>layer.kv_cache</code> (the tensor vLLM bound to that layer); positional fallback only when single-group                                             |
| <code>_skive_block_table</code>                                                                             | <code>input_batch.block_table[0]</code>                 | <code>input_batch.block_table[gid]</code> ; <code>None</code> if unresolved or hybrid block sizes                                                         |
| <code>_softmax_with_sink</code> ,<br><code>_layer_sinks</code>                                              | plain <code>torch.softmax</code>                        | sink-aware softmax from <code>layer.impl.sinks</code>                                                                                                     |
| <code>_agg_heads</code>                                                                                     | sum over heads                                          | sum / max / zmax via <code>SKIVE_HEAD_AGG</code>                                                                                                          |
| <code>_batch_score_va</code>                                                                                | one query, plain softmax, sum heads, bf16 cache assumed | per-layer own cache, sink-aware, chosen aggregation, optional fused Triton scorer, optional trajectory window, optional redundancy term, fp8 cache decode |
| <code>_batch_prescore</code> ,<br><code>_vk_score_cached</code> ,<br><code>score_request_blocks</code>      | walked every layer's cache                              | walk only the full group's caches ( <code>_skive_full_caches</code> ); fp8-aware                                                                          |
| <code>evict_request_blocks</code>                                                                           | fixed sink count                                        | <code>SKIVE_PROTECT_PROMPT</code> : sink covers the prompt                                                                                                |
| <code>skive_post_step</code>                                                                                | pairs <code>(req_id, j)</code>                          | triples <code>(req_id, j, gid)</code> ; optional query-history capture; step counter for cadence                                                          |
| <code>skive_reclaim</code>                                                                                  | <code>managers[0]</code> , pairs                        | <code>managers[gid]</code> , triples (pairs still accepted as group 0)                                                                                    |
| <code>_kv_blocks_f32</code>                                                                                 | did not exist                                           | decode uint8-stored fp8 blocks with the layer's scales                                                                                                    |
| <code>_skive_capture_qhist</code> ,<br><code>_score_multi_query</code>                                      | did not exist                                           | trajectory scoring window ( <code>SKIVE_QHIST</code> ), off by default                                                                                    |
| <code>_redundancy</code> , <code>_zscore_rows</code>                                                        | did not exist                                           | R-KV style redundancy penalty ( <code>SKIVE_REDUNDANCY</code> ), off by default                                                                           |

The scoring math in `_batch_score_va`, per sampled layer, for `R` over-budget rows padded to `max_nb` blocks (`T = max_nb * block_size` tokens): gather `K`, `V` for the rows' block ids into `[R, T, Hkv, D]` (fp8 decoded if needed), expand `K` to the query heads by `repeat_interleave(g)` with `g = Hq / Hkv = 8`, `logits = einsum("rhd,rhtd->rht", q, K) * scale`, mask padded tokens to `-inf`, `p = _softmax_with_sink(logits, sinks)`, `per_head = p * ||V||_1` (value\_attention) or `p` (h2o, snapkv), `token = _agg_heads(per_head)`, `block score = sum` over the block's tokens (max for snapkv), accumulated over layers. One `.tolist()` at the end is the only GPU-to-CPU synchronization of the step.

## 6.3 Other package files

`selection.py`: `plan_eviction` finds real (non-null) slots, excludes the first `sink` and last `local` of them, and caps the eviction count; `choose_lowest` is a stable argsort so ties go to the oldest block. `compaction.py`: section 5.3. `fused_attention.py`: the team's Triton kernel kept as provided, plus `skive_token_score_kernel`, a two-pass (max, then exp/sum), sink-aware scoring kernel with the sequence length as a runtime argument; used when `SKIVE_SCORER=triton`.

## 6.4 The harness changes for gpt-oss

`skive/run_gptoss.py` and `skive/gptoss_campaign.py`: force the attention backend (`SKIVE_ATTN_BACKEND=auto`: FlashAttention on Hopper, Triton elsewhere) so the run is deterministic and the log names the file the hooks live in; set `VLLM_USE_V2_MODEL_RUNNER=0`, `VLLM_USE_FLASHINFER_SAMPLER=0`, `VLLM_ALLOW_INSECURE_SERIALIZATION=1`; harmony final-channel extraction; `reasoning_effort` pass-through; `CUDAGRAPH=piecewise`; `KV_DTYPE`, `BLOCK_SIZE`, `--tag` variants; the eviction-rate columns; CSV export.

## 7. One decode step on gpt-oss with SKIVE, end to end

1. The scheduler picks the running requests and builds `CommonAttentionMetadata` per KV-cache group; the worker's `GPUModelRunner` prepares inputs.
2. For the full-attention group, `TritonAttentionMetadataBuilder.build` runs: if eviction is on and any decode row has a null slot, the block table and `seq_lens` for that group are compacted (edit K). The sliding-window group's builder leaves its table alone (`_skive_can_compact` is false).
3. The forward pass runs. In each full-attention layer, `TritonAttentionImpl.forward` stores the query on the layer (edit K) and calls `unified_attention` with the compacted table, so evicted keys are never read. Sinks are applied inside the kernel.
4. Sampling produces the step's tokens; `ModelRunnerOutput` is built.
5. `skive_post_step` runs (edit E): resolves the full group (cached), counts real blocks per row, defers unless it is an eviction step or a row is far over budget, optionally captures the query history, scores the over-budget rows in one batched call (sink-aware, `zmax`, fp8-aware), nulls the lowest-scoring candidates outside the protected sink (prompt) and local regions, and records `(req_id, j, gid)` triples.
6. Back in the scheduler process (edits F/G), the triples are pulled through `collective_rpc` and each block is freed in `single_type_managers[gid]`; the pool's free count rises, and the next request can be admitted.
7. The next step starts at 1 with a smaller gather list for the evicting requests.

## 8. Verification

- **Unit tests (`skive/tests/`, run by the installer and by CI):** selection planning and stable ties; compaction on NumPy and torch, including the decode-only rule; multi-group resolution prefers the pure full-attention group, falls back to a windowed `FullAttentionSpec`, resolves through `attn_groups` when the config is absent, and refuses on an unresolvable model; reclaim frees in the evicted group and never in the sliding-window group; the hybrid-block guard; sink-aware softmax equals an explicit reference and preserves within-head ranking; `zmax` equals normalize-then-max; the fused scorer matches the torch path (GPU only); the Triton edit K applies once to a stub carrying the real anchors and to the shipped file, compiles, compacts decode rows only, respects the sliding-window and CUDA-graph guards, and captures the query only for query-dependent metrics; multi-query scoring reduces to the single-query path for a window of one and averages over the window; the redundancy term flags duplicates; fp8 blocks decode and scale. 74 tests pass on the box; the CI workflow runs them on CPU.
- **On-box evidence (L40S, gpt-oss-20b):** Using `AttentionBackendEnum.TRITON_ATTN backend.;`  
`[SKIVE DBG] ... gid=1` (the full-attention group is group 1, not 0); `[SKIVE 4c] evicted ... cumulative=336` with matching `[SKIVE 4c-ii] freed ... pool_free=` lines rising as blocks return to the pool; `evicted_blocks=336` reported through the RPC counter.
- **Reproducibility:** the patcher applied all 17 edits to pristine v0.23.0 and produced files byte-identical to the validated overlay.
- **Results:** AIME 2024, all 30 problems, 8 samples: FullKV 68.8; SKIVE at 4096 tokens 72.1 with 56 percent of KV reclaimed; at 2048 tokens 63.8 with 78 percent reclaimed and 17 percent less wall time at 240 concurrent traces. Full tables in `skive/results/aime24/`.

## 9. Limitations and open items

- Sparse-gather on the Triton path is disabled under full CUDA-graph capture; piecewise graphs (the recommended mode) and eager mode are unaffected. FlashInfer (the default on SM100 GPUs) has no hooks; force `TRITON_ATTN` there.
- Prefix caching must be off (reclaim ignores shared-block reference counts).
- Per-layer budgets are not expressible because a KV-cache group shares one block table across its layers; only the group boundary gives per-layer behaviour (which is exactly how the sliding-window layers are excluded).
- FP8 KV cache combined with eviction gave inconsistent AIME numbers (61.2 and 68.3 at 4096); FP8 for FullKV is fine. The likely cause is quantization noise in the value norms used by the score.
- Trajectory scoring and the redundancy penalty are implemented, tested and off by default: no gain on AIME, a small gain on HotpotQA.